

# JavaScript Interview Topics

A complete roadmap from basic to advanced concepts

## 1. Basic Level

### Language Fundamentals

- Variables: var vs let vs const, hoisting, scope differences
- Data types: primitives (string, number, boolean, null, undefined, symbol, bigint) vs reference types
- Type conversion and coercion (implicit vs explicit)
- Operators: arithmetic, comparison (== vs ===), logical, ternary, nullish coalescing (??), optional chaining (?.)
- Template literals and string methods
- Control flow: if/else, switch, loops (for, while, do-while, for...in, for...of)

### Functions

- Function declarations vs function expressions vs arrow functions
- Parameters, default parameters, rest parameters
- Arguments object
- Return values and function scope
- Immediately Invoked Function Expressions (IIFE)

### Objects & Arrays

- Object literals, property access (dot vs bracket notation)
- Array basics: creation, indexing, common methods (push, pop, shift, unshift, slice, splice)
- Iterating objects and arrays (for...in, for...of, Object.keys/values/entries)
- Array methods: map, filter, reduce, forEach, find, some, every
- Destructuring assignment (objects and arrays)
- Spread and rest operators

### DOM Basics

- Selecting elements (getElementById, querySelector, querySelectorAll)
- DOM manipulation (innerHTML, textContent, createElement, appendChild)
- Event handling basics (addEventListener, event object)
- Basic form handling and validation

## 2. Intermediate Level

### Scope & Closures

- Lexical scope and scope chain
- Closures: definition, use cases, common interview problems
- Block scope vs function scope

- The 'this' keyword: rules in different contexts (global, object method, arrow function)

## Asynchronous JavaScript

- Call stack, event loop, task queue, microtask queue
- Callbacks and callback hell
- Promises: creation, chaining, Promise.all/race/allSettled/any
- Async/await syntax and error handling with try/catch
- setTimeout, setInterval, and their pitfalls

## Object-Oriented JavaScript

- Prototypes and the prototype chain
- Constructor functions vs ES6 classes
- Inheritance: prototypal inheritance, extends/super keywords
- Object.create(), Object.assign(), getPrototypeOf
- Encapsulation techniques (closures, private fields with #)

## Error Handling & Debugging

- try/catch/finally, custom error objects
- Throwing and propagating errors
- Debugging techniques and browser dev tools

## Working with the Browser

- Event bubbling, capturing, and delegation
- Event object properties and preventDefault/stopPropagation
- Browser storage: localStorage, sessionStorage, cookies
- Fetch API and XMLHttpRequest basics
- JSON parsing and stringifying

# 3. Advanced Level

## Advanced Functions & Patterns

- Higher-order functions and function composition
- Currying and partial application
- Debounce and throttle implementation
- Memoization
- Design patterns: Module, Singleton, Factory, Observer, Pub/Sub

## Advanced Asynchronous Concepts

- Event loop deep dive: microtasks vs macrotasks execution order
- Generators and iterators (function\*, yield)
- Async iterators and for-await-of
- Web Workers basics

- Race conditions and concurrency control in async code

## Memory & Performance

- Garbage collection and memory leaks
- Performance optimization techniques (reflow/repaint, lazy loading)
- Understanding call stack overflow and tail call optimization

## Modules & Tooling

- ES modules (import/export) vs CommonJS (require/module.exports)
- Bundlers basics: Webpack, Vite, Rollup (conceptual understanding)
- Transpilation with Babel and why it's needed
- Package management: npm/yarn, package.json, semantic versioning

## Advanced Language Internals

- Execution context and the call stack in detail
- Hoisting: var, let/const (temporal dead zone), function hoisting
- Symbol type and its use cases
- Proxy and Reflect objects
- WeakMap and WeakSet vs Map and Set
- Tagged template literals
- Object property descriptors (writable, enumerable, configurable)

## Testing & Best Practices

- Unit testing basics (Jest, Mocha concepts)
- Writing clean, maintainable, DRY code
- Common security issues: XSS, CSRF basics in JS context

## Framework-Adjacent Concepts (often asked)

- Virtual DOM concept (React-related but often asked generally)
- State management concepts
- Component lifecycle concepts (if applying for frontend roles)

## 4. Common Coding Interview Problems

- Reverse a string / array without built-in methods
- Flatten a nested array
- Implement debounce and throttle from scratch
- Deep clone an object
- Implement Array.prototype.map/filter/reduce manually
- Find duplicates in an array
- Implement a simple Promise from scratch
- Check for palindrome, anagram
- Implement currying function

- Polyfill for bind/call/apply

## 5. Interview Preparation Tips

- Practice explaining concepts out loud, not just coding them
- Understand the 'why' behind concepts (e.g., why closures exist, why event loop matters)
- Be ready to write code on a whiteboard or shared editor without autocomplete
- Review real-world use cases for each concept, not just definitions
- Practice on platforms like LeetCode, HackerRank, or JavaScript30